Introdução às Redes Neurais Artificiais

Aula 09 — Do neurônio à MLP

Neurônio • Ativação • MLP • Backpropagation

Prof. Luis Vasconcelos | Duração: 100 min

Das Árvores às Curvas Suaves

Árvores (RF, XGBoost)

- Dividem o espaço em regiões
- Aproximação por DEGRAUS
- Excelente para dados de processo
- Uma superfície suave precisaria de muitos degraus

Redes Neurais

- Aproximam CURVAS suaves
- Diferenciáveis (gradiente)
- Aproximador universal de funções
- Podem capturar relações complexas

"E se a relação entre features e target for uma curva suave e complexa?"

Pergunta Desafio

"Um neurônio só consegue aprender relações LINEAIRES.
Como 3 neurônios em camada aprendem uma curva?
Onde nasce a não-linearidade?"

💡 A resposta: a função de ATIVAÇÃO não-linear em cada neurônio.

O Neurônio que Aprende

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b \qquad a = \sigma(z)$$

Analogia: neurônio = válvula de controle na planta

⚖️ Pesos (w)

Importância de cada feature
maior w = maior impacto

🔧 Viés (b)

Limiar de ativação
"quanto precisa somar"

⚡ Ativação σ(z)

Decide se o neurônio 'liga'
introduz a não-linearidade

📤 Saída (a)

Passa para a próxima camada
ou é a predição final

Funções de Ativação

ReLU: $a = \max(0, z)$

Padrão p/ camadas ocultas
Barata, evita gradiente desaparecendo

Sigmoid: $a = 1/(1+e^{-z})$

Saída entre 0 e 1
Classificação binária

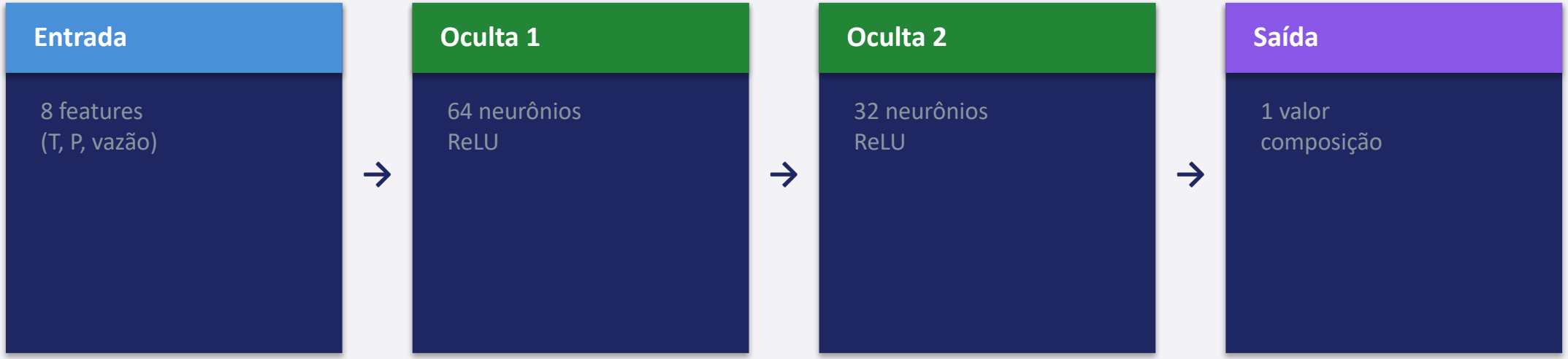
Tanh: $a = \tanh(z)$

Saída [-1,1]
Dados centrados

Linear: $a = z$

Regressão (saída)
Sem ativação final

MLP — Multilayer Perceptron



Feedforward: o sinal flui da entrada → oculta → saída ($x \rightarrow h_1 \rightarrow h_2 \rightarrow \hat{y}$)

$$\hat{y} = f_{out}(W_2 \cdot f_{oculta}(W_1 \cdot x + b_1) + b_2)$$

Backpropagation — A Regra da Cadeia

- O erro na saída é uma função COMPOSTA
- Erro = $L(\hat{y}(h_2(h_1(x))))$
- A regra da cadeia diz quanto cada peso contribuiu
- Ajustamos cada peso na direção que REDUZ o erro
- $\Delta w_{ij} = -\eta \cdot \partial Erro/\partial w_{ij}$ (gradiente descendente)
- A 'mágica' é cálculo diferencial em cadeia

Aproximador Universal: uma MLP com 1 camada oculta + ativação não-linear pode aproximar qualquer função contínua.

Pipeline MLP — 5 Passos

①	**Arquitetura**	`MLPRegressor((64, 32), activation='relu')`
②	**Escalar**	`StandardScaler()` — fit no treino, transform no teste
③	**Treinar**	`model.fit(X_train_scaled, y_train)`
④	**Predizer**	`model.predict(X_test_scaled)`
⑤	**Avaliar**	`RMSE, R², predicted vs real`

Arquitetura da MLP — 4 Grupos

Grupo A

((16,))

activation: relu

1) RMSE treino/teste
2) Curva de perda
3) Mais parâmetros = melhor?

Grupo B

((64, 32))

activation: relu

1) RMSE treino/teste
2) Curva de perda
3) Mais parâmetros = melhor?

Grupo C

((256, 128))

activation: relu

1) RMSE treino/teste
2) Curva de perda
3) Mais parâmetros = melhor?

Grupo D

((64, 32))

activation: tanh

1) RMSE treino/teste
2) Curva de perda
3) Mais parâmetros = melhor?

🧠 **Desafio: Por que MLP (256,128,64) pode ter teste PIOR que (64,32)?**

Tarefa: MLP vs XGBoost para o CSTR

- Use o dataset CSTR (features + conversão)
- Treine MLP (64, 32, 16) + ReLU com dados escalados
- Treine XGBoost (200) como baseline
- Compare RMSE, R^2 e TEMPO de treinamento (%%time)
- Plote predicted vs real (lado a lado)
- Quando escolher MLP vs XGBoost?
- Entregue via Colab — Prazo: antes da Aula 10 (7 dias)

Bônus NT: MLP com 1 camada de (2, 8, 32, 128) neurônios — existe ponto de retorno decrescente? (+20%)

Próxima aula: Lab Prático 2 — treinar MLPs com Keras/TensorFlow! 🚀